

Ejercicio 1

Se desea obtener un algoritmo de orden logarítmico que, dada una lista ordenada de enteros y otro entero, devuelva la posición del primer elemento de la lista que sea mayor o igual que dicho entero. En caso de que no exista, es decir, todos los elementos de la lista son menores, el algoritmo debe devolver el tamaño de la lista. Pueden existir elementos repetidos.

```
public static Integer ejercicio1(List<Integer> lista, Integer a)
```

SE PIDE:

- a. Defina la función recursiva que representa el algoritmo explicado anteriormente.
- b. Proporcione una solución recursiva.
- c. Proporcione una solución iterativa usando while.
- d. Proporcione una solución en notación funcional.

SOLUCIÓN:**a) Definición:**

$$f(l, a, i, j) = \begin{cases} j & \text{si } i == j \\ f(l, a, i, \frac{i+j}{2} + 1, j) & \text{si } l[\frac{i+j}{2}] < a \\ f(l, a, i, \frac{i+j}{2}) & \text{e.o.c} \end{cases}$$

b) Solución recursiva:

```
public static int primero(List<Integer> lista, Integer a, int i, int j) {
    int pos = -1;
    if (i==j) {
        pos = j;
    } else {
        int k = (i+j)/2;
        if (lista.get(k)<a) {
            pos = primero(lista, a, k+1, j);
        } else {
            pos = primero(lista, a, i, k);
        }
    }
    return pos;
}
```

c) Solución iterativa:

```
public static int primeroIt(List<Integer> lista, Integer a, int i, int j) {
    int pos = -1;
    while (!(i==j)) {
        int k = (i+j)/2;
        if (lista.get(k)<a) {
            i = k+1;
        } else {
            j = k;
        }
    }
    pos = j;
    return pos;
}
```

d) Solución funcional:

```
private static record Tupla (List<Integer> lista, Integer a, int i, int j) {
    public static Tupla of(List<Integer> lista, Integer a, int i, int j){
        return new Tupla(lista, a, i,j);
    }

    public static Tupla first(List<Integer> lista, Integer a) {
        return of(lista, a, 0, lista.size());
    }

    public Tupla next() {
        Tupla t = null;
        int k = (i+j)/2;
        if (lista.get(k)<a) {
            t = of(lista,a,k+1,j);
        } else {
            t = of(lista,a,i,k);
        }
        return t;
    }
}

public static int primeroFnc(List<Integer> lista, Integer a, int i, int j) {
    Tupla t = Stream.iterate(Tupla.first(lista,a),e -> e.next())
        .filter(e ->e.i()==e.j()).findFirst().get();
    return t.j();
}
```